UNIT 6
Graphs

Outline

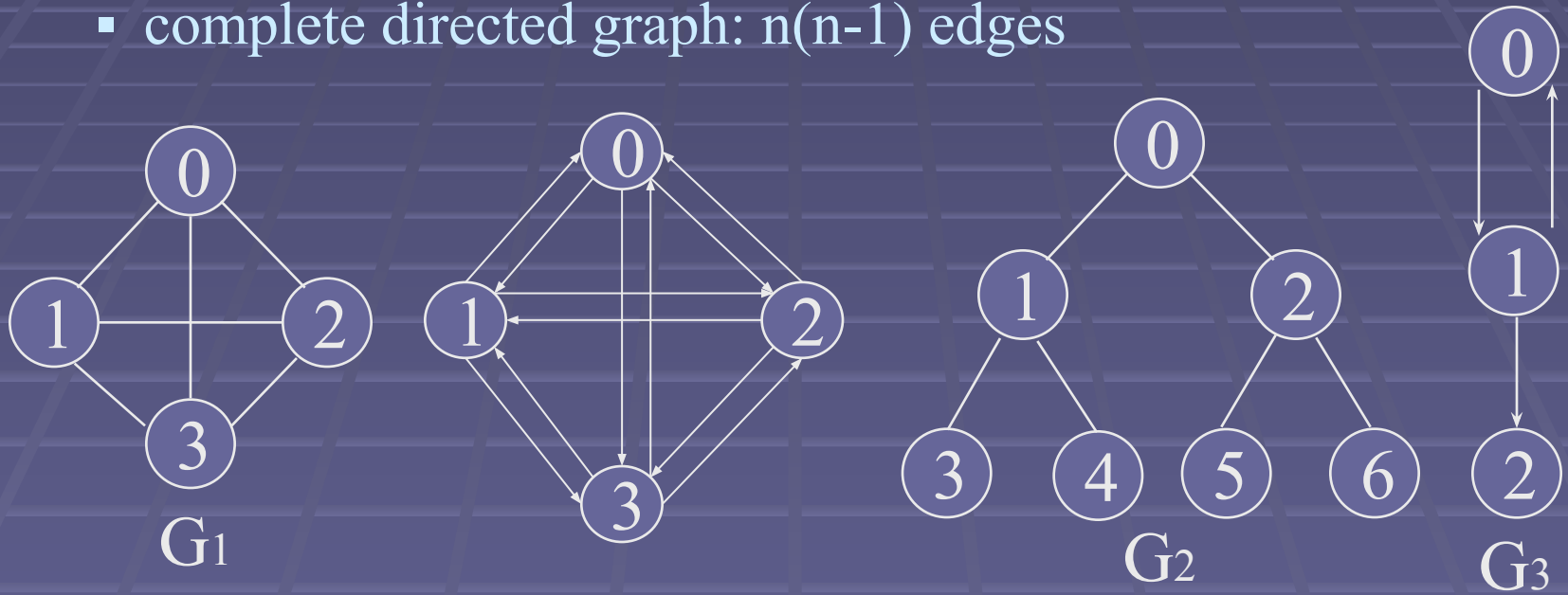
- Basic concepts of graph theory
- Storage representation and manipulation of graphs
- Introduction to Sparse matrix
- Representation of sparse matrix using linked list

Graph

- Definitions
  - A graph G consists of two sets
    - a finite, nonempty set of vertices $V(G)$
    - a finite, possible empty set of edges $E(G)$
  - $G(V,E)$ represents a graph
  - An undirected graph is one in which the pair of vertices in an edge is unordered, $(v0, v1) = (v1,v0)$
  - A directed graph is one in which each edge is a directed pair of vertices, $<v0, v1> != <v1, v0>$



- Examples for Graph
  - complete undirected graph: n(n-1)/2 edges
  - complete directed graph: n(n-1) edges



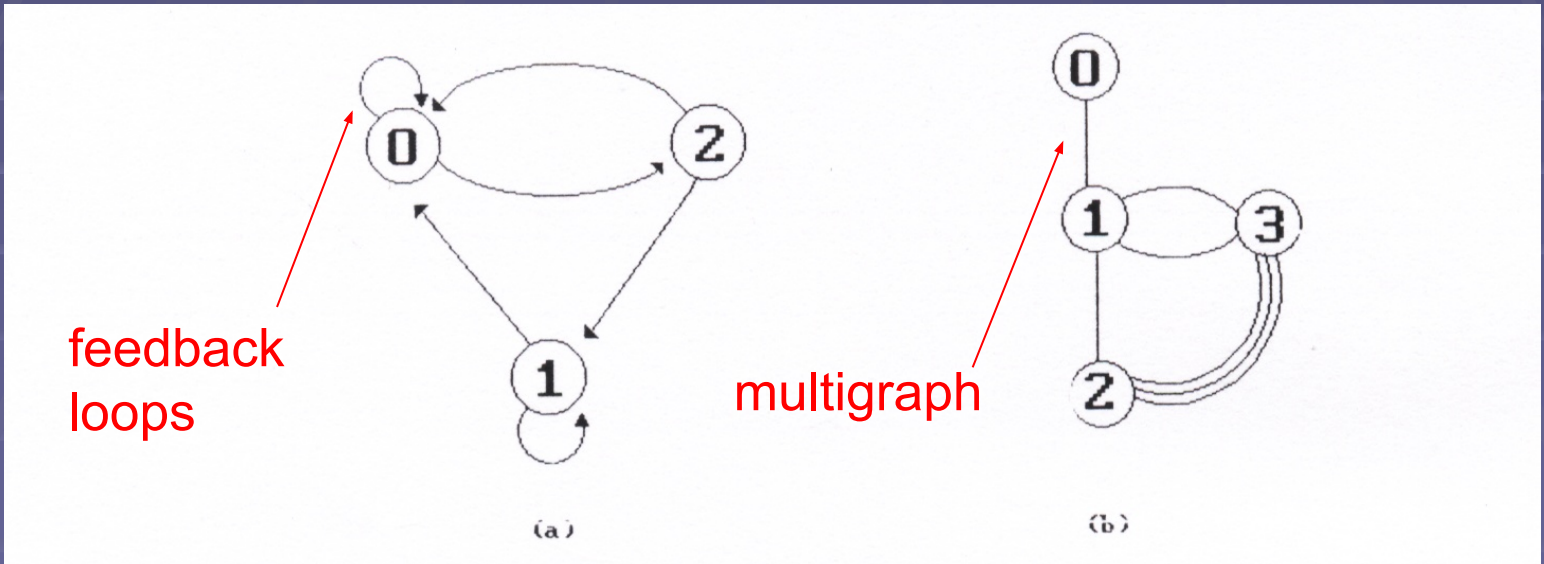
complete graph

incomplete graph

$V(G_1)=\{0,1,2,3\}$
$V(G_2)=\{0,1,2,3,4,5,6\}$
$V(G_3)=\{0,1,2\}$

$E(G_1)=\{(0,1),(0,2),(0,3),(1,2),(1,3),(2,3)\}$
$E(G_2)=\{(0,1),(0,2),(1,3),(1,4),(2,5),(2,6)\}$
$E(G_3)=\{<0,1>,<1,0>,<1,2>\}$

- Restrictions on graphs
  - A graph may not have an edge from a vertex, *i*, back to itself. Such edges are known as *self loops*
  - A graph may not have multiple occurrences of the same edge. If we remove this restriction, we obtain a data referred to as a multigraph



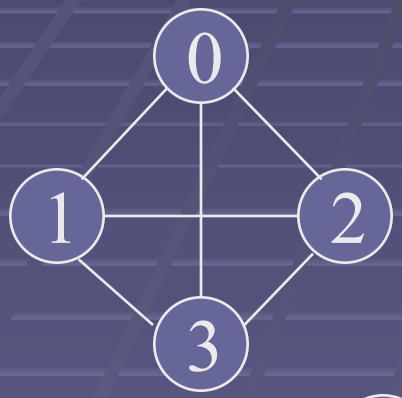
- Adjacent and Incident
- If ($v0$, $v1$) is an edge in an undirected graph,
  - $v0$ and $v1$ are adjacent
  - The edge ($v0$, $v1$) is incident on vertices $v0$ and $v1$



- If <$v0$, $v1$> is an edge in a directed graph
  - $v0$ is adjacent to $v1$, and $v1$ is adjacent from $v0$
  - The edge <$v0$, $v1$> is incident on $v0$ and $v1$



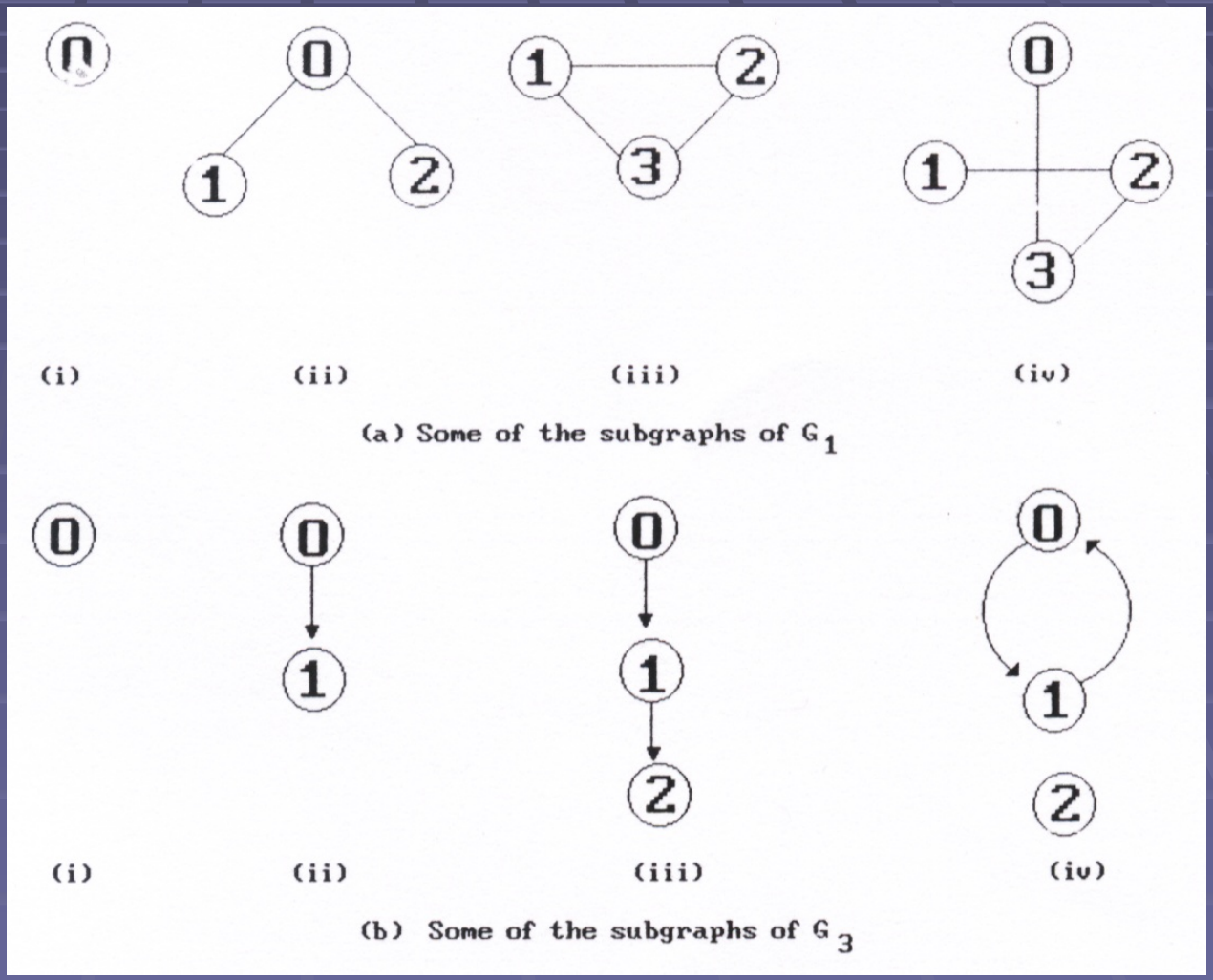
- A subgraph of *G* is a graph *G*' such that V(*G*') $\subseteq$ V(*G*) and E(*G*') $\subseteq$ E(*G*).



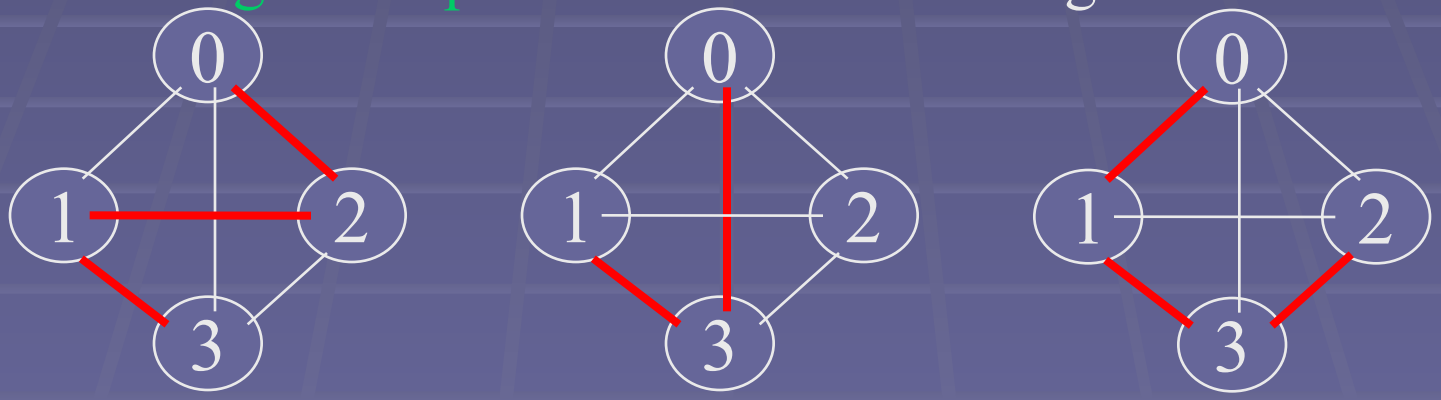
G_1



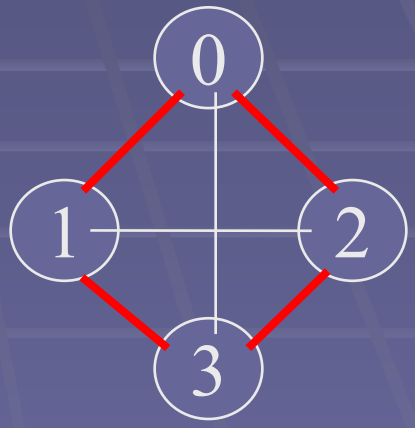
G_3



- Path
  - A path from vertex *vp* to vertex *vq* in a graph *G*, is a sequence of vertices, *vp*, *vi*1, *vi*2, ..., *vi*n, vq, such that (*vp*, *vi*1), (*vi*1, *vi*2), ..., (*vi*n, v*q*) are edges in an undirected graph.
    - A path such as (0, 2), (2, 1), (1, 3) is also written as 0, 2, 1, 3
  - The length of a path is the number of edges on it



- Simple path and cycle
  - simple path (simple directed path): a path in which all vertices, except possibly the first and the last, are distinct.
  - A cycle is a simple path in which the first and the last vertices are the same.



- Connected graph
  - In an undirected graph G, two vertices, v_0 and v_1, are connected if there is a path in G from v_0 to v_1
  - An undirected graph is connected if, for every pair of distinct vertices v_i, v_j, there is a path from v_i to v_j
- Connected component
  - A connected component of an undirected graph is a maximal connected subgraph.
  - A tree is a graph that is connected and acyclic *(i.e, has no cycle)*.

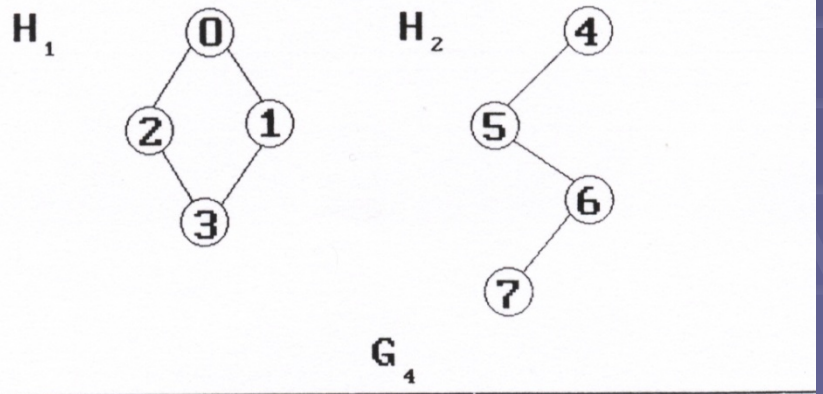


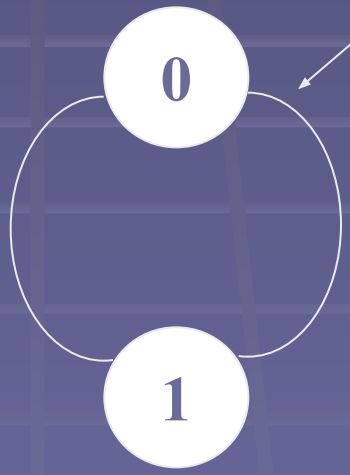
Figure 6.5: A graph with two connected components

- Strongly Connected Component
  - A directed graph is strongly connected if there is a directed path from *vi* to *vj* and also from *vj* to *vi*
  - A strongly connected component is a maximal subgraph that is strongly connected



G_3 not strongly connected

strongly connected component
(maximal strongly connected subgraph)





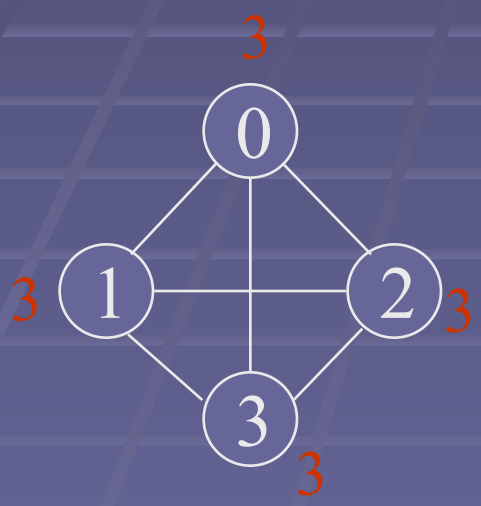
- Degree
  - The degree of a vertex is the number of edges incident to that vertex.
- For directed graph
  - in-degree (v) : the number of edges that have v as the head
  - out-degree (v) : the number of edges that have v as the tail
- If di is the degree of a vertex i in a graph G with n vertices and e edges, the number of edges is

$$e = (\sum_{0}^{n-1} d_i) / 2$$

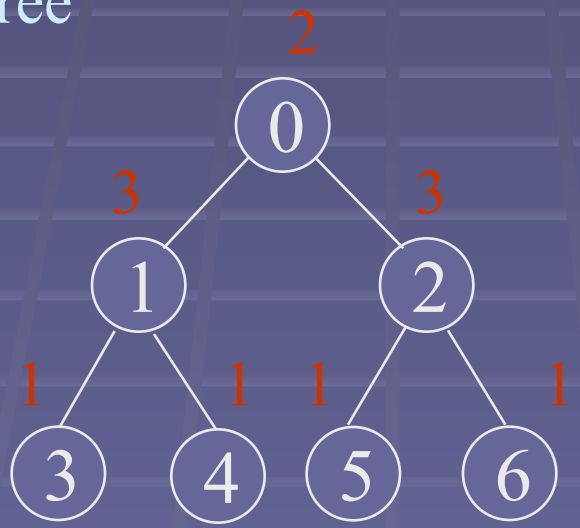
- Degree (cont'd)
- We shall refer to a directed graph as a *digraph*. When we us the term *graph*, we assume that it is an undirected graph

undirected graph

degree



G_1



G_2

directed graph

in-degree & out-degree



in:1, out: 1

in: 1, out: 2

in: 1, out: 0

G_3

Graph ADT

- Abstract Data Type *Graph*

structure *Graph* is

objects: a nonempty set of vertices and a set of undirected edges, where each edge is a pair of vertices.

functions:

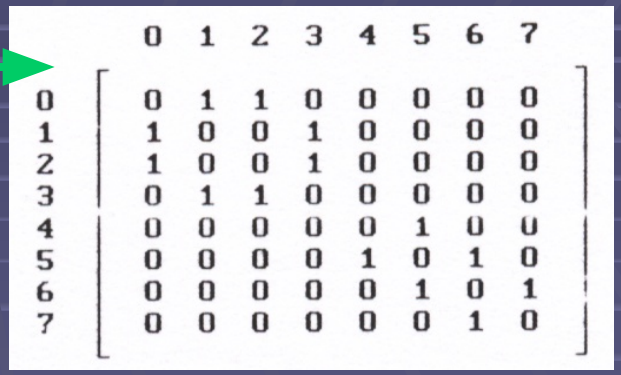
for all *graph* $\in$ *Graph*, v, v_1, and $v_2 \in$ *Vertices*

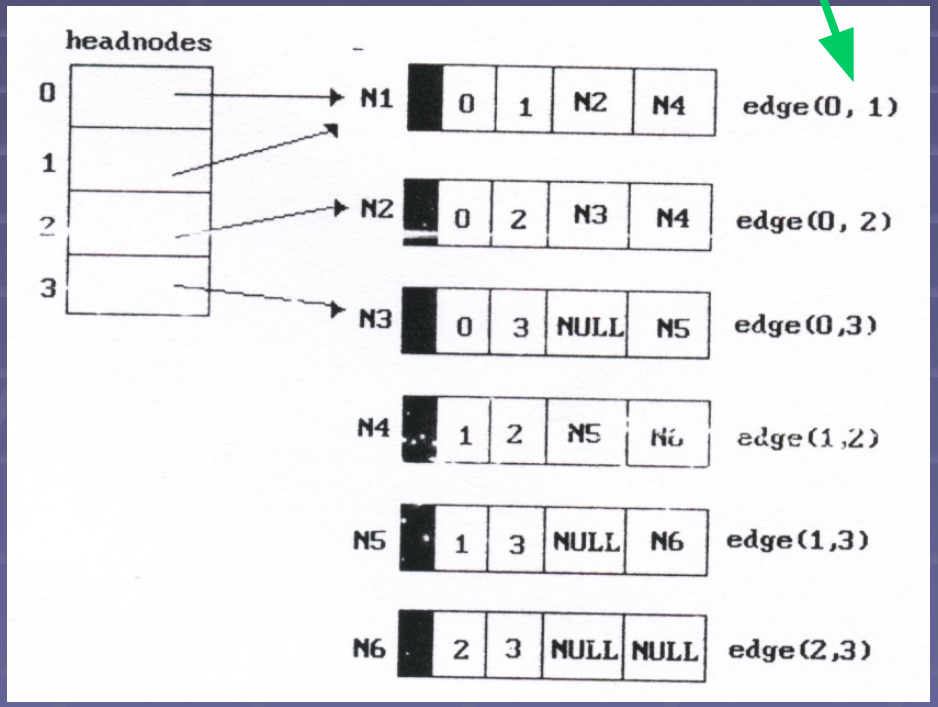
Graph Create()	::=	**return** an empty graph.
Graph InsertVertex(*graph*, v)	::=	**return** a graph with v inserted. v has no incident edges.
Graph InsertEdge(*graph*, v_1, v_2)	::=	**return** a graph with a new edge between v_1 and v_2.
Graph DeleteVertex(*graph*, v)	::=	**return** a graph in which v and all edges incident to it are removed.
Graph DeleteEdge(*graph*, v_1, v_2)	::=	**return** a graph in which the edge (v_1, v_2) is removed. Leave the incident nodes in the graph.
Boolean IsEmpty(*graph*)	::=	**if** (*graph* == empty graph) **return** *TRUE* **else return** *FALSE*.
List Adjacent(*graph*, v)	::=	**return** a list of all vertices that are adjacent to v.

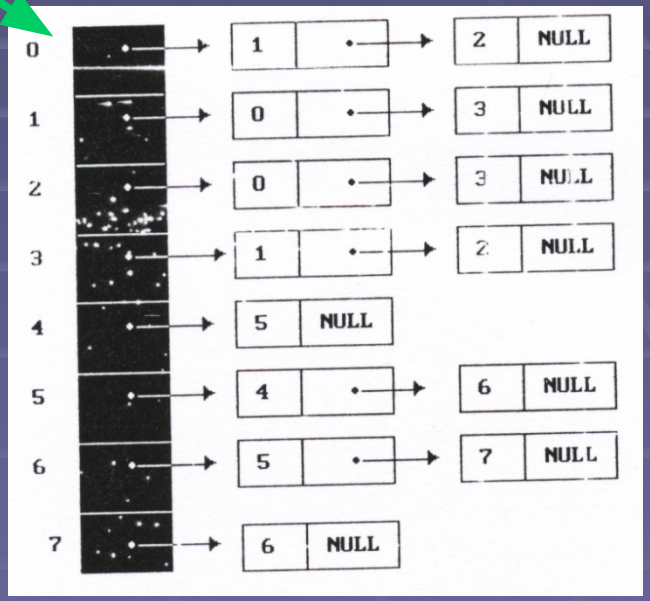
Structure 6.1: Abstract data type *Graph*

Graph Representations

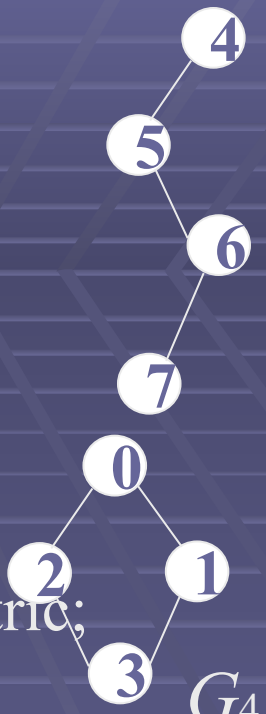
- Adjacency Matrix
- Adjacency Lists
- Adjacency Multilists



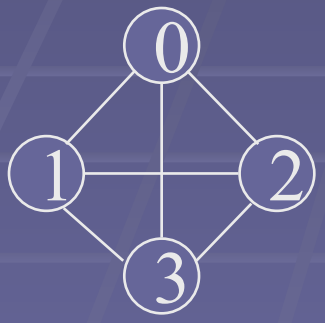




- Adjacency Matrix
  - Let $G = (V,E)$ be a graph with n vertices.
  - The adjacency matrix of G is a two-dimensional n x n array, say *adj_mat*
  - If the edge (vi, vj) is(not) in $E(G)$, *adj_mat[i][j]*=1(0)
  - The adjacency matrix for an undirected graph is symmetric; the adjacency matrix for a digraph need not be symmetric



G_4



$$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

G_1



$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

G_3

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

- Merits of Adjacency Matrix
  - For an undirected graph, the degree of any vertex, *i*, is its row sum:

$$\sum_{j=0}^{n-1} adj_mat[i][j]$$

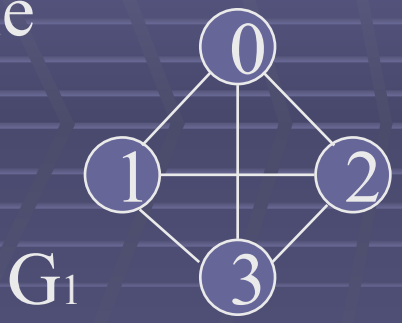
  - For a directed graph, the row sum is the out-degree, while the column sum is the in-degree.

$$ind(vi) = \sum_{j=0}^{n-1} A[j,i] \quad outd(vi) = \sum_{j=0}^{n-1} A[i,j]$$

  - The complexity of checking edge number or examining if *G* is connect
    - *G* is undirected: $O(n^2/2)$
    - *G* is directed: $O(n^2)$

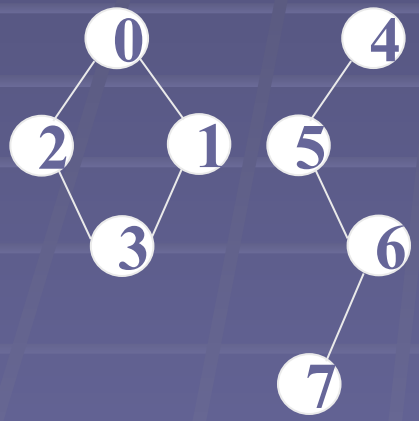
- Adjacency lists
  - There is one list for each vertex in G. The nodes in list i represent the vertices that are adjacent from vertex i
  - For an undirected graph with n vertices and e edges, this representation requires n head nodes and $2e$ list nodes
  - *C* declarations for adjacency lists

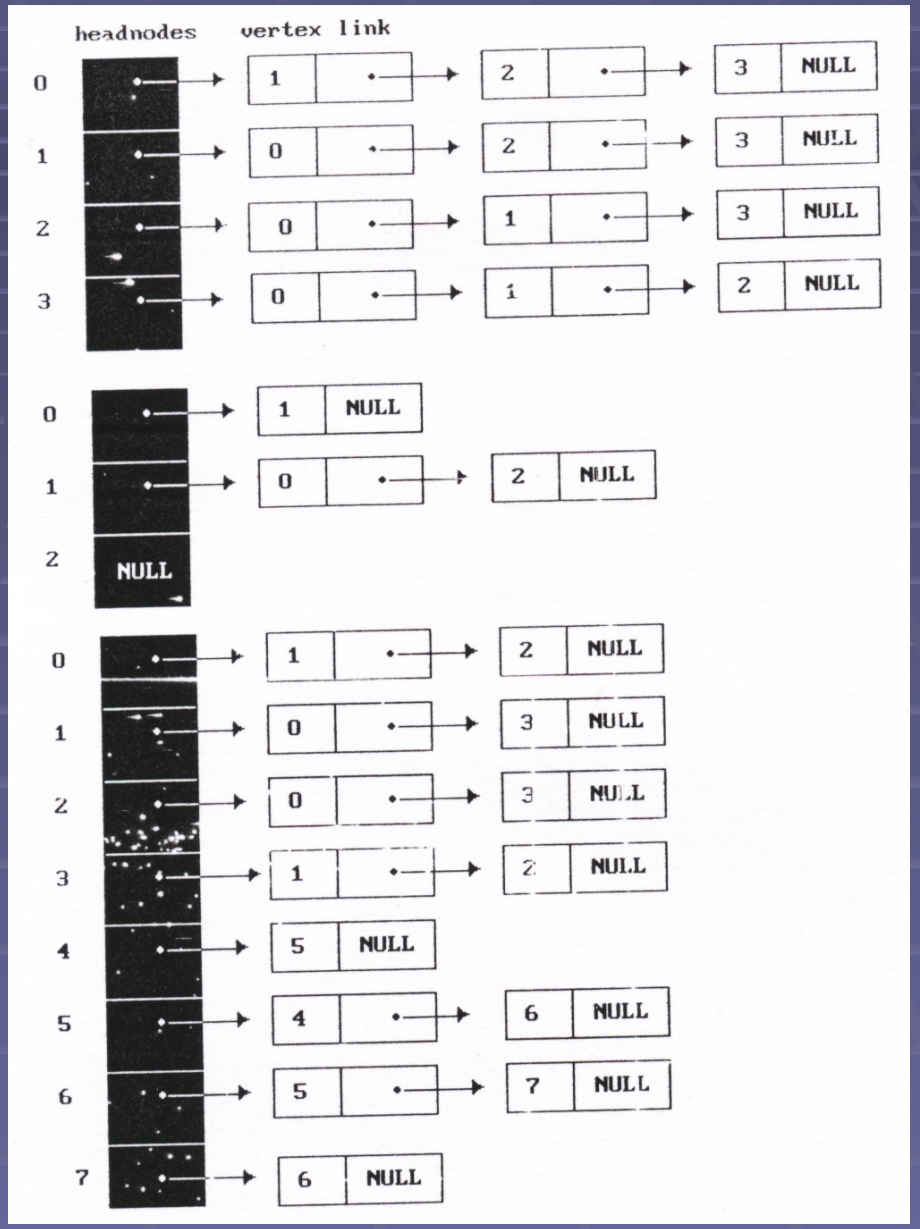
- Example



G_1

G_4





G_3

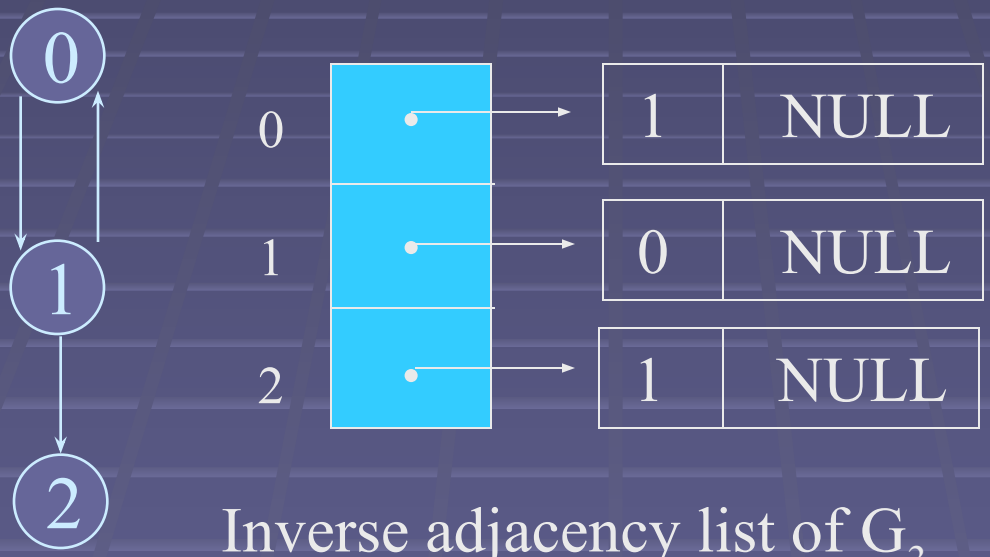


- Interesting Operations
  - degree of a vertex in an undirected graph
    - # of nodes in adjacency list
  - # of edges in a graph
    - determined in O(n+e)
  - out-degree of a vertex in a directed graph
    - # of nodes in its adjacency list
  - in-degree of a vertex in a directed graph
    - traverse the whole data structure

- Sequential Representation of Graph
  - Sequentially pack the nodes on the adjacency lists
  - *node*[1] ~ *node*[*n*+2*e*+1] may be used.
  - *node*[0] ~ *node*[*n*-1] gives the starting point of the list for vertex *i*
  - *node*[*n*]: *n*+2*e*+1
  - *node*[*n*+1] … *node*[*n*+2*e*]: head node of edge
  - The vertices adjacent from vertex *i* are stored in *node*[*node*[*i*]], … , *node*[*node*[*i*+1]-1], $0 \leqq i < n$
  - The number of edge in *G* may be determined in O(*n*+*e*) time

[0]	9		[12]	3	
[1]	11		[13]	0	2
[2]	13		[14]	3	
[3]	15		[15]	1	3
[4]	17		[16]	2	
[5]	18		[17]	5	4
[6]	20		[18]	4	5
[7]	22		[19]	6	
[8]	23		[20]	5	6
[9]	1	0	[21]	7	
[10]	2		[22]	6	7
[11]	0	1			

- Finding In-degree of Vertices

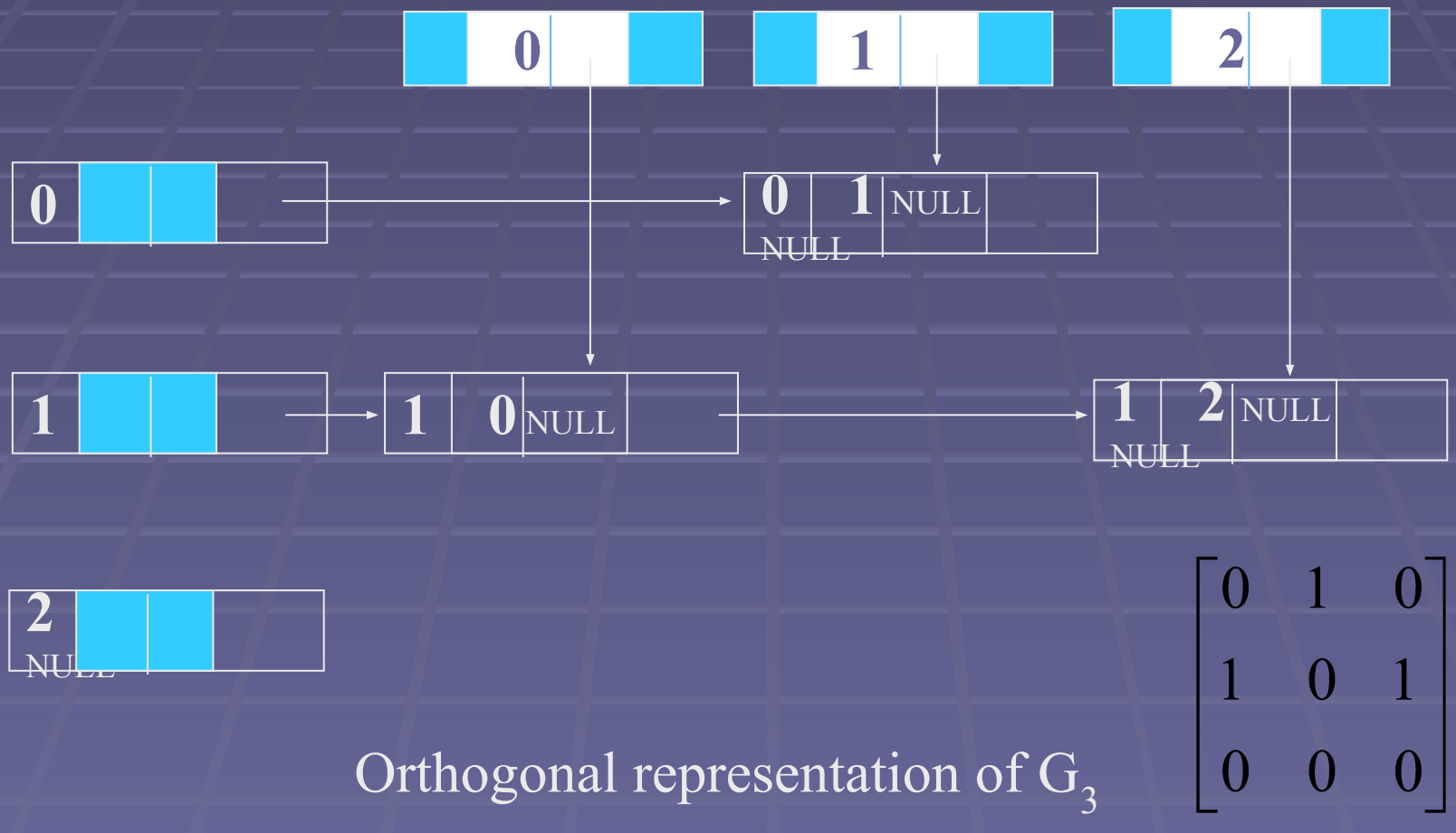


Inverse adjacency list of G_3

tail	head	column link for head	row link for tail

Alternate node structure of adjacency lists

- Example of Changing Node Structure

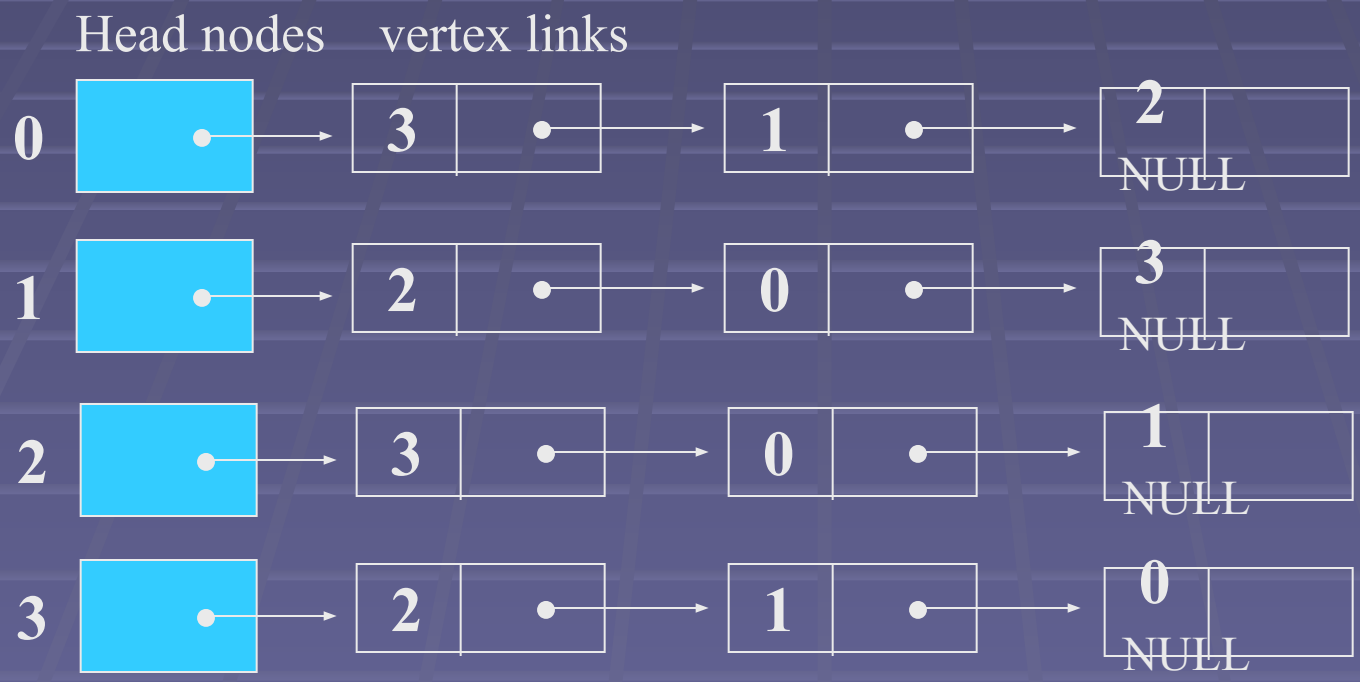


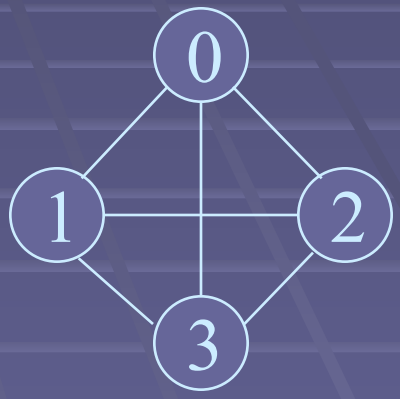
Orthogonal representation of G_3

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

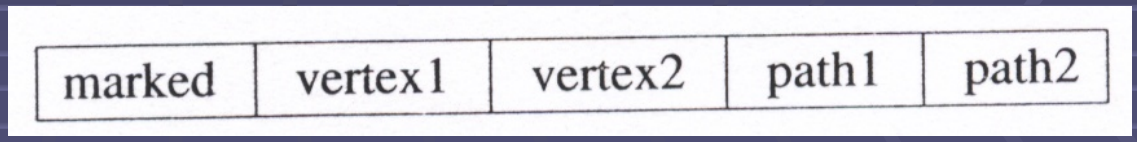
- Vertices in Any Order

Order is of no significance



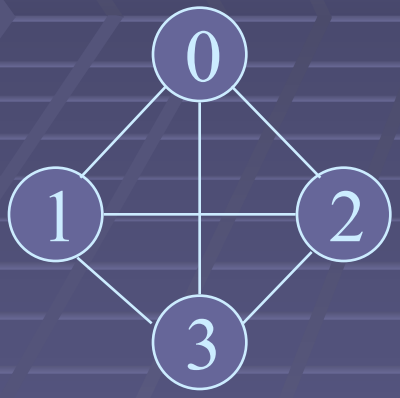


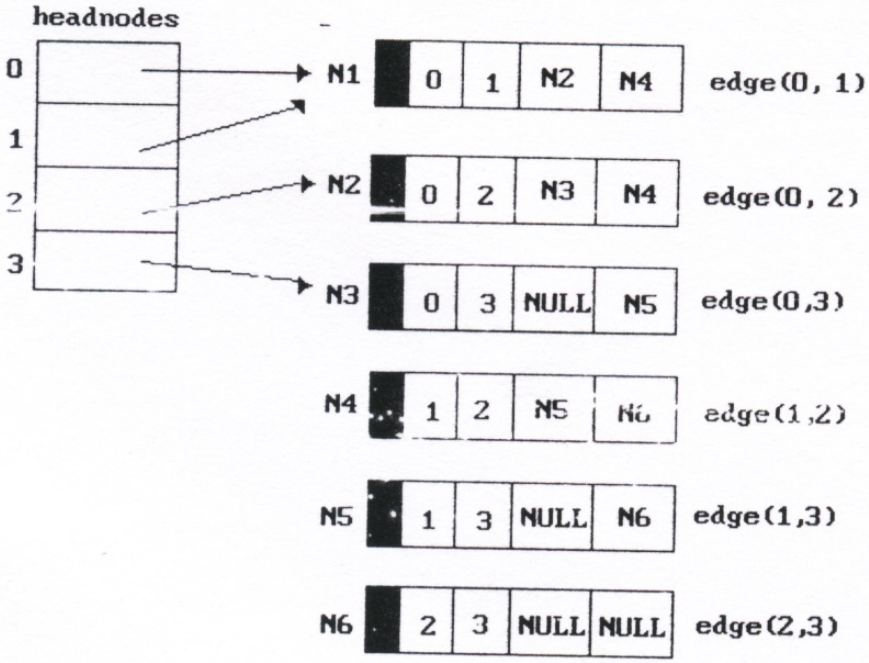
- Adjacency Multilists
  - Lists in which nodes may be shared among several lists. (an edge is shared by two different paths)
  - There is exactly one node for each edge.
  - This node is on the adjacency list for each of the two vertices it is incident to




```
typedef struct edge *edge_pointer;
typedef struct edge {
        short int marked;
        int vertex1;
        int vertex2;
        edge_pointer path1;
        edge_pointer path2;
        };
edge_pointer graph[MAX_VERTICES];
```

- Example for Adjacency Multlists





The lists are: vertex 0: M1 → M2 → M3
vertex 1: M1 → M4 → M5
vertex 2: M2 → M4 → M6
vertex 3: M3 → M5 → M6

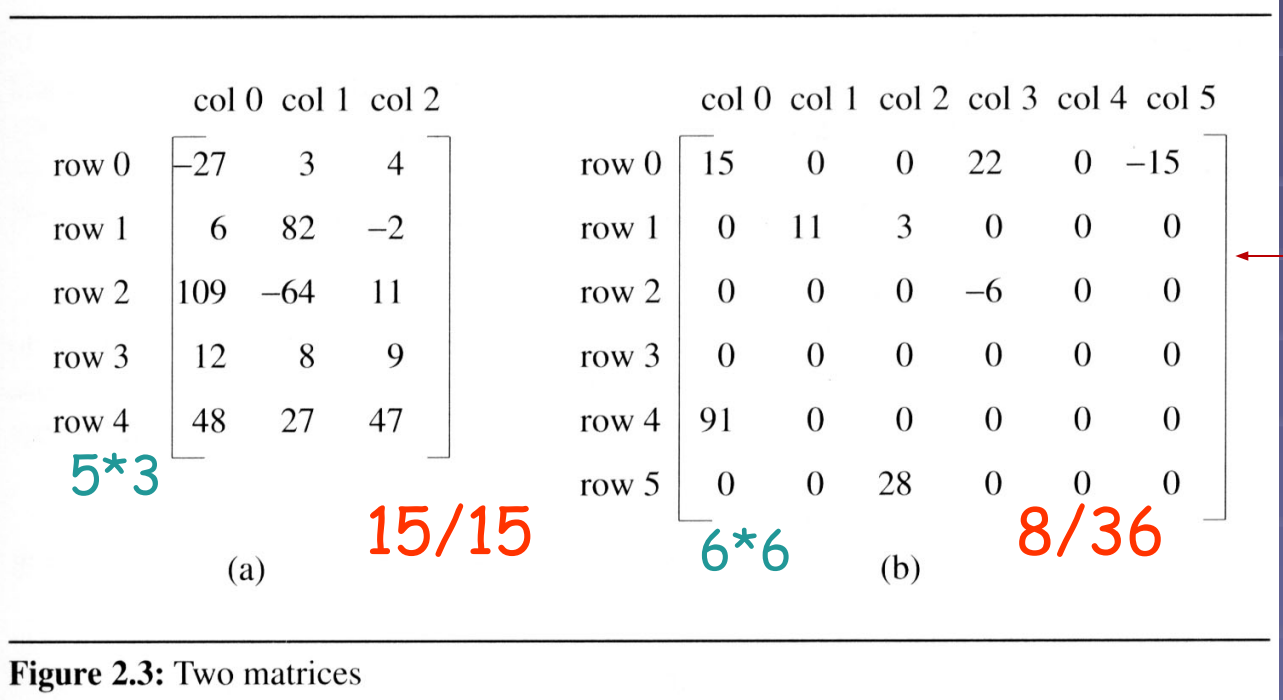
Figure 6.15: Adjacency multilists for G_1

- Weighted edges
  - The edges of a graph have weights assigned to them.
  - These weights may represent as
    - the distance from one vertex to another
    - cost of going from one vertex to an adjacent vertex.
  - adjacency matrix: *adj_mat*[*i*][*j*] would keep the weights.
  - adjacency lists: add a *weight* field to the node structure.
  - A graph with weighted edges is called a *network*

Lecture No.40

Sparse matrix

1. Introduction
   - In mathematics, a matrix contains *m* rows and *n* columns of elements, we write *m*×*n* to designate a matrix with *m* rows and *n* columns.



	col 0	col 1	col 2
row 0	−27	3	4
row 1	6	82	−2
row 2	109	−64	11
row 3	12	8	9
row 4	48	27	47

5*3

15/15

(a)

	col 0	col 1	col 2	col 3	col 4	col 5
row 0	15	0	0	22	0	−15
row 1	0	11	3	0	0	0
row 2	0	0	0	−6	0	0
row 3	0	0	0	0	0	0
row 4	91	0	0	0	0	0
row 5	0	0	28	0	0	0

6*6

8/36

(b)

sparse matrix data structure?

Figure 2.3: Two matrices

- The standard representation of a matrix is a two dimensional array defined as

  a[*MAX_ROWS*][*MAX_COLS*]

  - We can locate quickly any element by writing *a*[*i*][*j*]
- Sparse matrix wastes space
  - We must consider alternate forms of representation.
  - Our representation of sparse matrices should store only nonzero elements.
  - Each element is characterized by <row, col, value>.

- Figure 2.4(a) shows how the sparse matrix of Figure 2.3(b) is represented in the array *a*.
  - Represented by a two-dimensional array.
  - Each element is characterized by <row, col, value>.

of rows (columns) # of nonzero terms

	row	col	value
a[0]	6	6	8
[1]	0	0	15
[2]	0	3	22
[3]	0	5	−15
[4]	1	1	11
[5]	1	2	3
[6]	2	3	−6
[7]	4	0	91
[8]	5	2	28

(a)

transpose

	row	col	value
b[0]	6	6	8
[1]	0	0	15
[2]	0	4	91
[3]	1	1	11
[4]	2	1	3
[5]	2	5	28
[6]	3	0	22
[7]	3	2	−6
[8]	5	0	−15

(b)

Figure 2.4: Sparse matrix and its transpose stored as triples

row, column in ascending order

2. Transpose a Matrix

- For each row i
  - take element <i, j, value> and store it in element <j, i, value> of the transpose.
  - difficulty: where to put <j, i, value>

    (0, 0, 15) ====> (0, 0, 15)
    (0, 3, 22) ====> (3, 0, 22)
    (0, 5, -15) ====> (5, 0, -15)
    (1, 1, 11) ====> (1, 1, 11)

    Move elements down very often.
- For all elements in column j,
  - place element <i, j, value> in element <j, i, value>

- This algorithm is incorporated in transpose (Program 2.7).

Assign
A[i][j] to B[j][i]

place element <i, j, value>
in element <j, i, value>

For all columns i
For all elements in column j

Scan the array
"columns" times.
The array has
"elements" elements.

```
void transpose(term a[], term b[])
/* b is set to the transpose of a */
{
   int n,i,j, currentb;
   n = a[0].value;        /* total number of elements */
   b[0].row = a[0].col; /* rows in b = columns in a */
   b[0].col = a[0].row; /* columns in b = rows in a */
   b[0].value = n;
   if (n > 0 )  { /* non zero matrix */
      currentb = 1;
      for (i = 0; i < a[0].col; i++)
      /* transpose by the columns in a */
         for (j = 1; j <= n; j++)
         /* find elements from the current column */
            if (a[j].col == i) {
            /* element is in current column, add it to b */
               b[currentb].row = a[j].col;
               b[currentb].col = a[j].row;
               b[currentb].value = a[j].value;
               currentb++;
            }
   }
}
```

==> O(columns*elements)

Program 2.7: Transpose of a sparse matrix

3. Matrix multiplication

- Definition:

  Given A and B where A is $m \times n$ and B is $n \times p$, the product matrix D has dimension $m \times p$. Its $\langle i, j \rangle$ element is

$$d_{ij} = \sum_{k=0}^{n-1} a_{ik} b_{kj}$$

  for $0 \leq i < m$ and $0 \leq j < p$.

- Example:

$$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Figure 2.5: Multiplication of two sparse matrices

- Sparse Matrix Multiplication
  - Definition: $[D]_{m*p}=[A]_{m*n} * [B]_{n*p}$
  - Procedure: Fix a row of A and find all elements in column j of B for j=0, 1, …, p-1.
  - Alternative 1.
    Scan all of B to find all elements in j.
  - Alternative 2.
    Compute the transpose of B.
    (Put all column elements consecutively)
    - Once we have located the elements of row i of A and column j of B we just do a merge operation similar to that used in the polynomial addition of 2.2

Lecture No.42

Representation of sparse matrix using linked list

Chain Representation

Node structure.



Single Chain

4

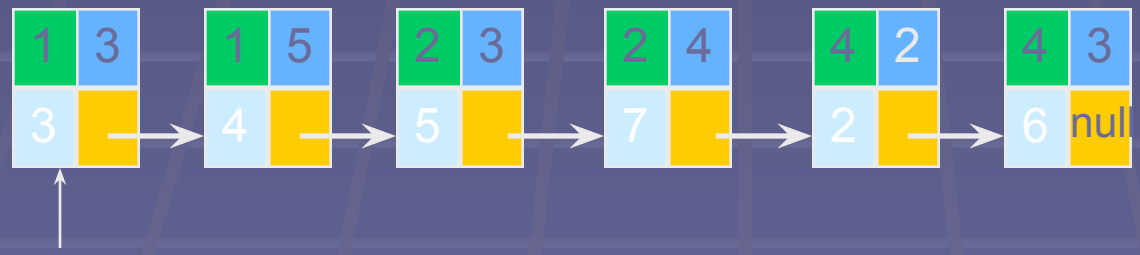
list = 3

row = [1 1 2 2 4]

column = [3 5 3 4 2]

value = 3 4 5 7 2

6



Traversal of graph

-DFS

-BFS (Breadth-first search)

Breadth-first search

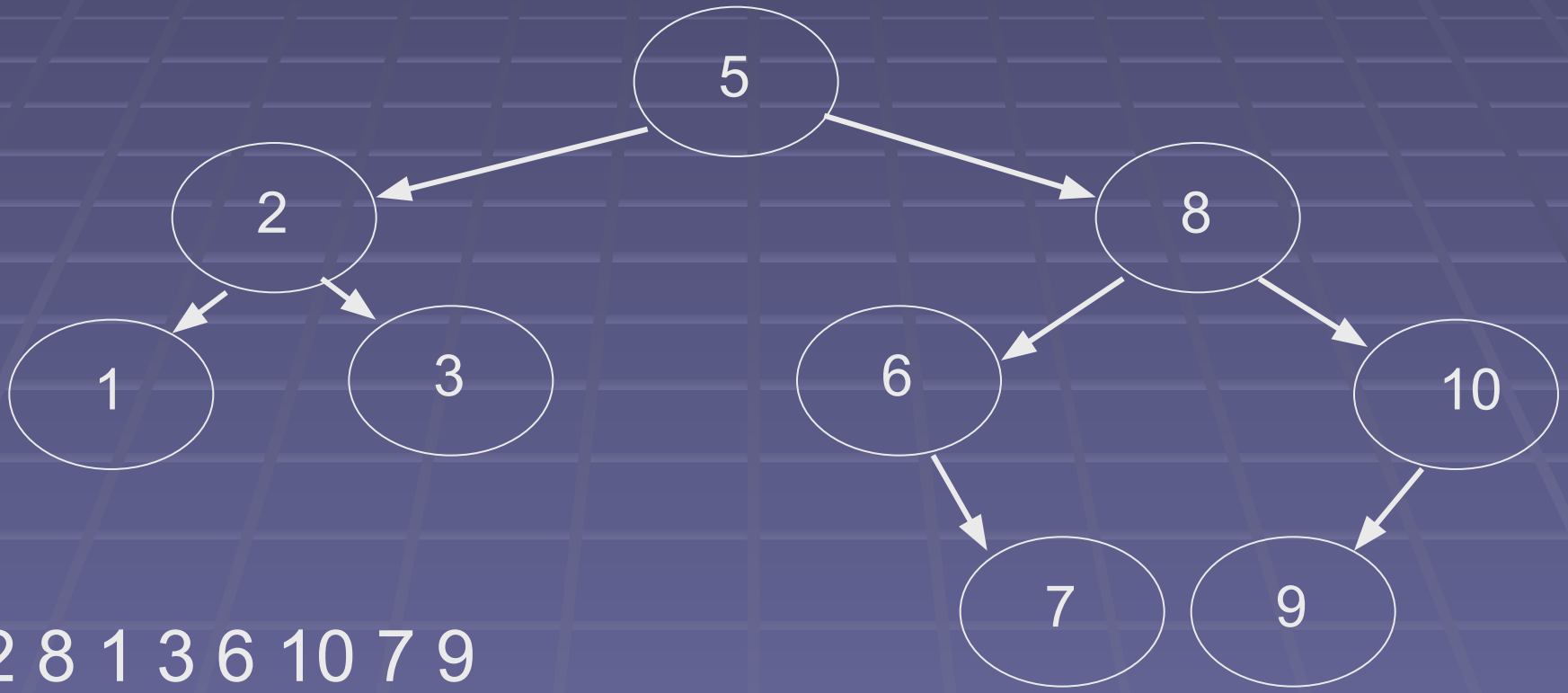
- One of the simplest algorithms
- Also one of the most important
  - It forms the basis for MANY graph algorithms

BFS: Level-by-level traversal

- Given a starting vertex s
- Visit all vertices at increasing distance from s
  - Visit all vertices at distance k from s
  - Then visit all vertices at distance k+1 from s
  - Then ….

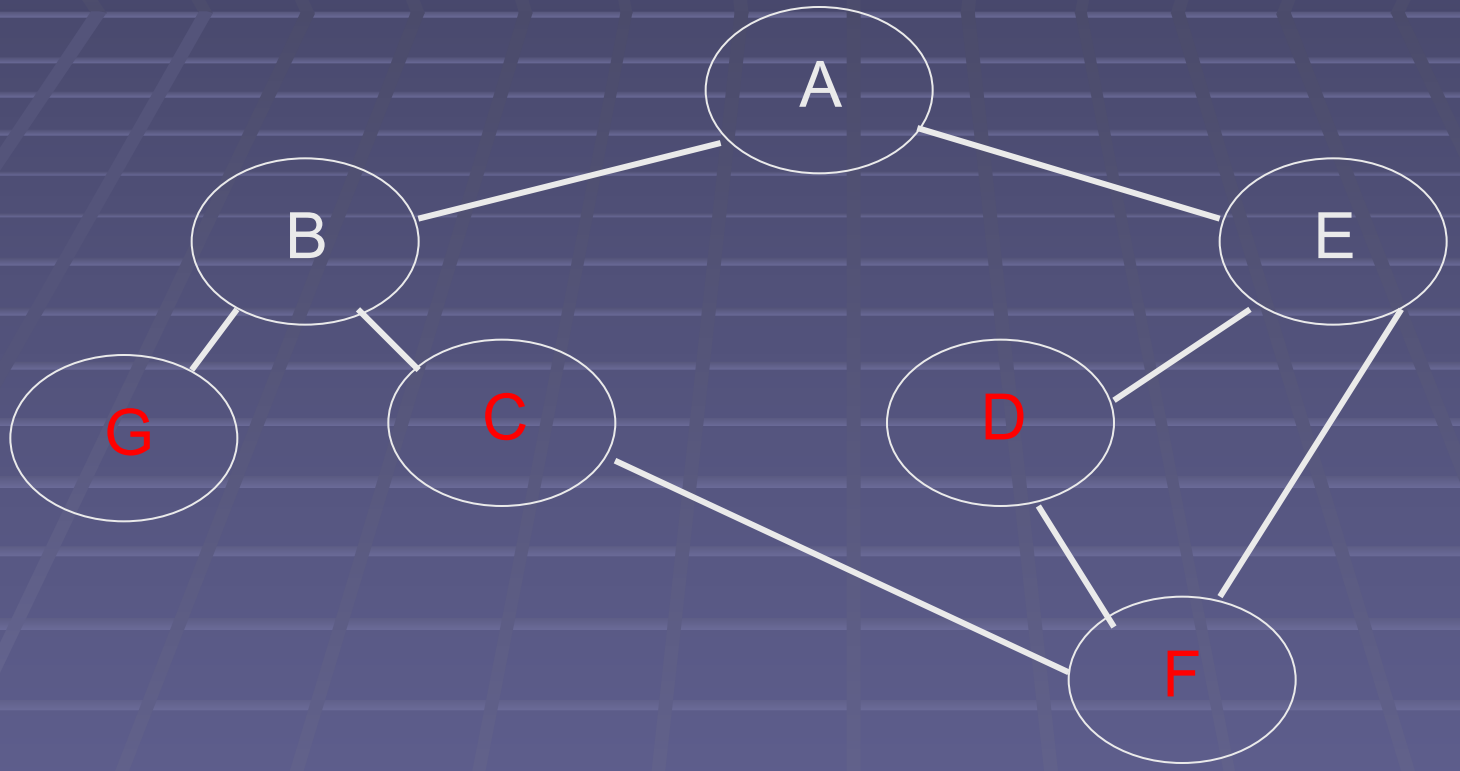
BFS in a binary tree (reminder)

BFS: visit all siblings before their descendents



5 2 8 1 3 6 10 7 9

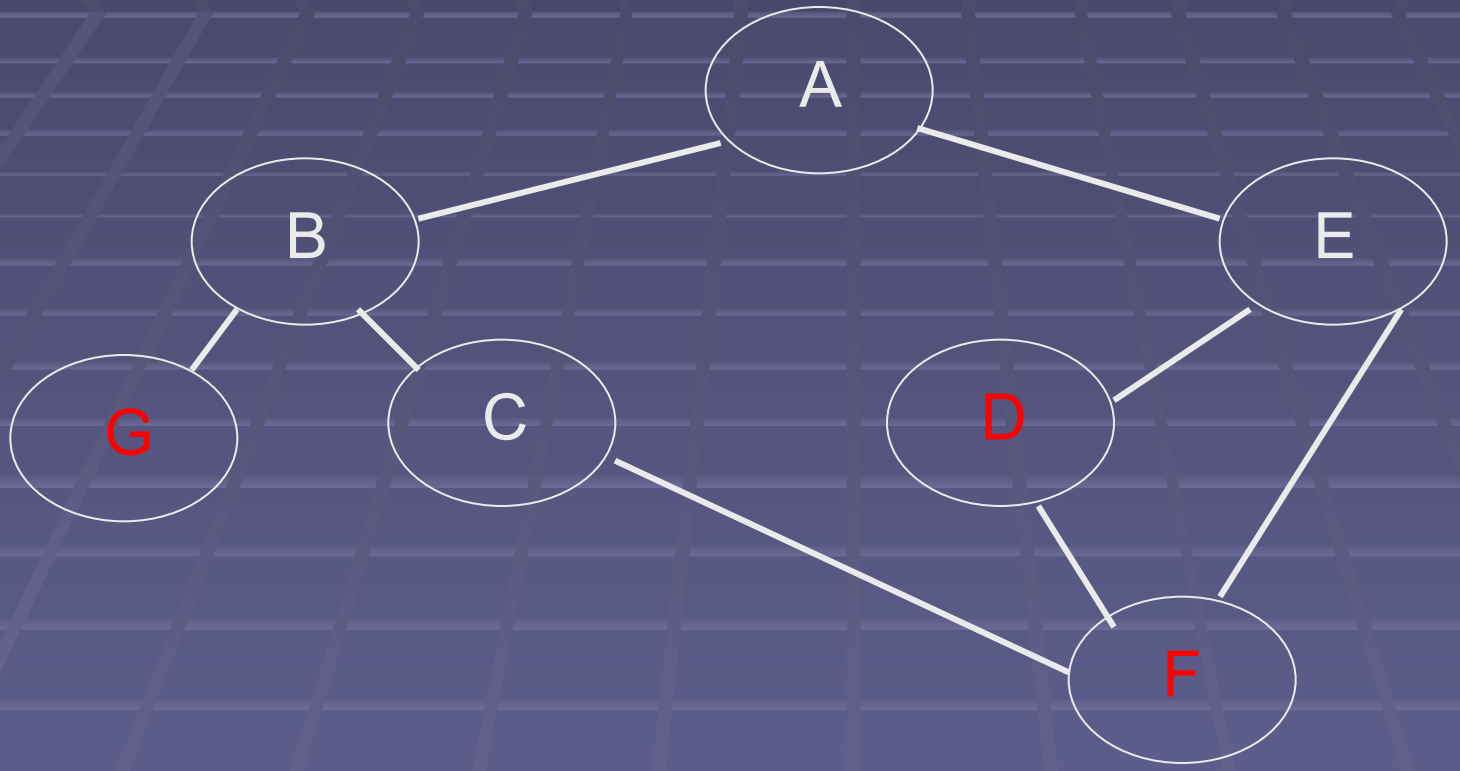
Queue: A B E C G D F



When we go to B, we put G and C in the queue

When we go to E, we put D and F in the

Queue: A B E C G D F F



Suppose we now want to expand C.

We put F in the queue again!

Depth-first search

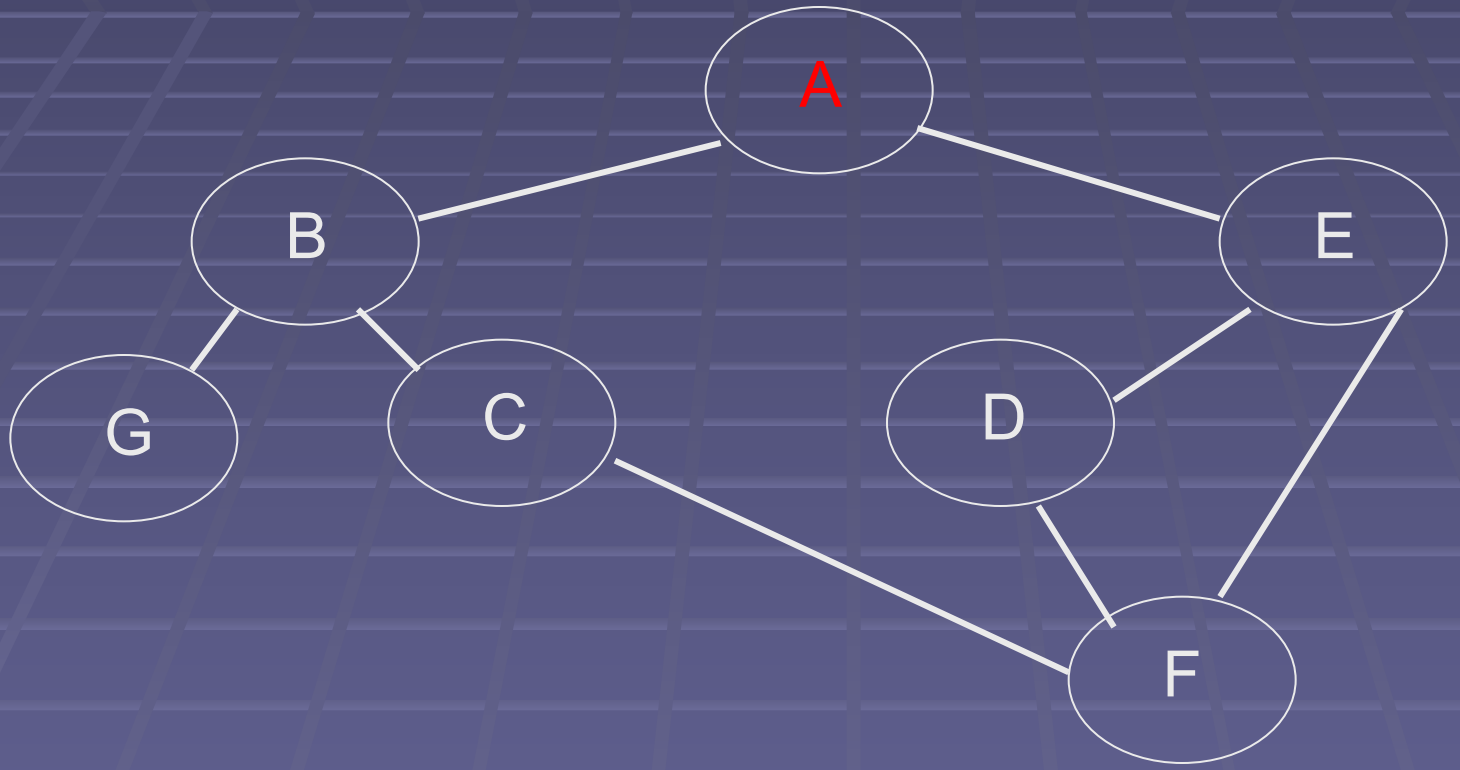
- Again, a simple and powerful algorithm
- Given a starting vertex s
- Pick an adjacent vertex, visit it.
  - Then visit one of its adjacent vertices
  - …..
  - Until impossible, then backtrack, visit another

DFS(graph g, vertex s)

Assume all vertices initially unmarked

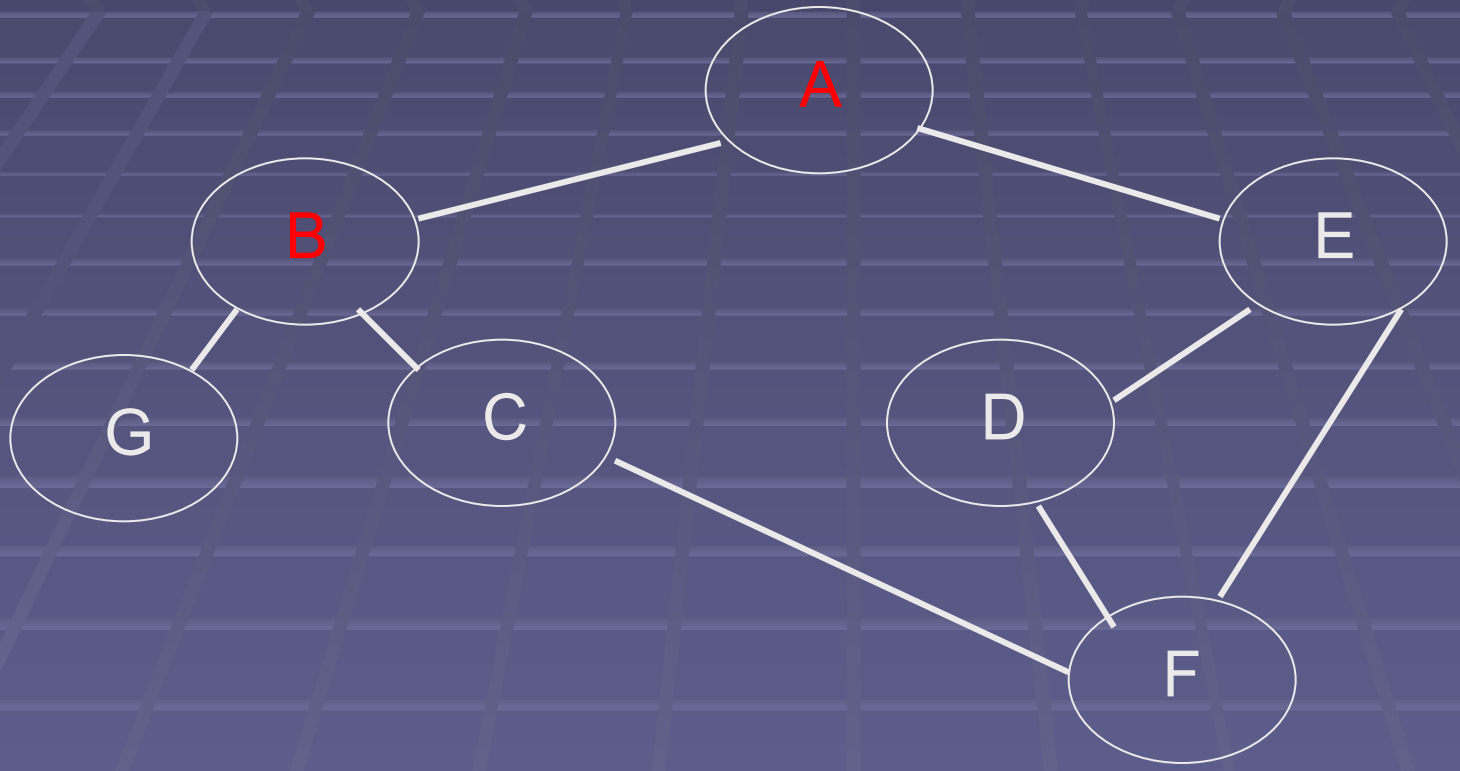
```
1. mark s
2. visit s // e.g., print its data
3. for each edge <s, V>
4.     if V is not marked
5.         DFS(G, V)
```

Current vertex: A



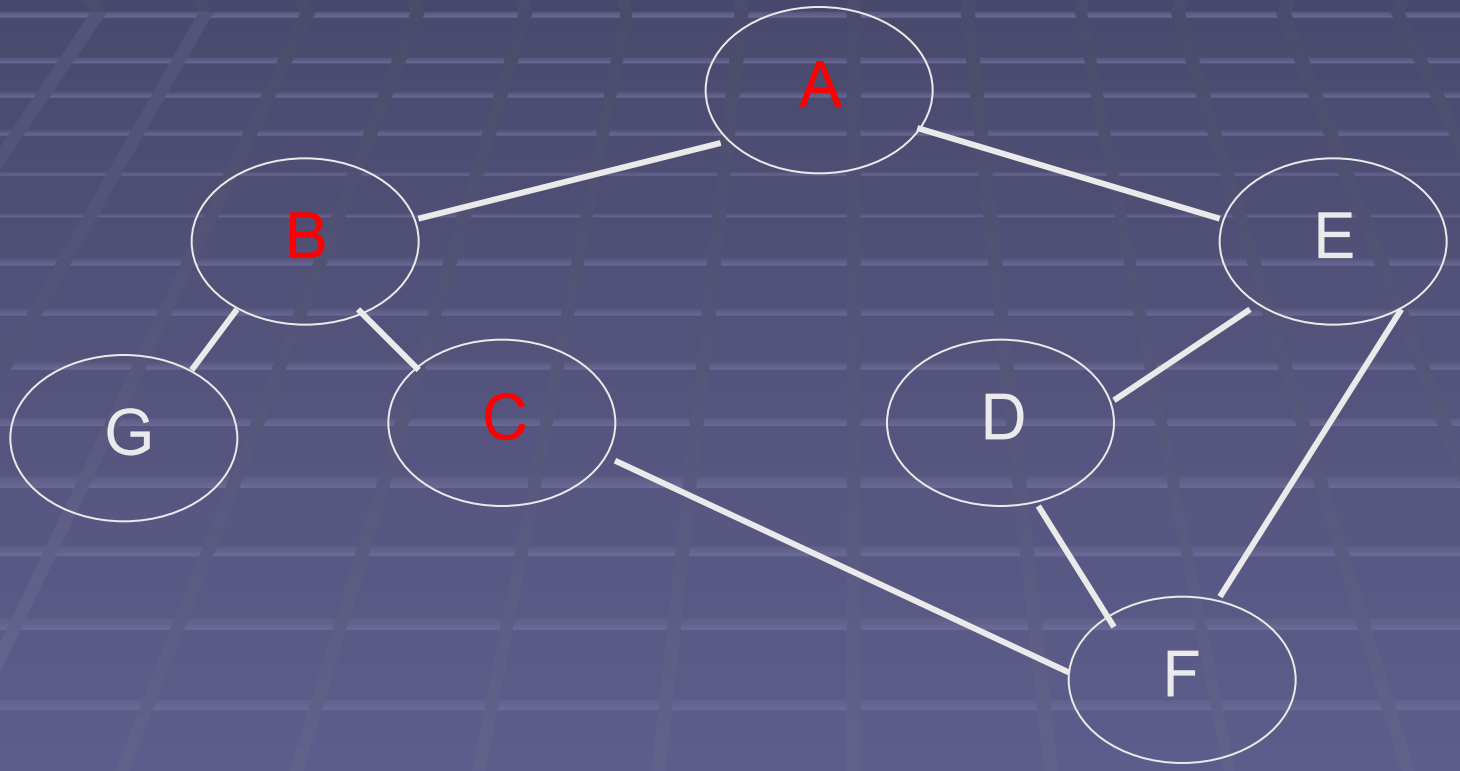
Start with A. Mark it.

Current: B



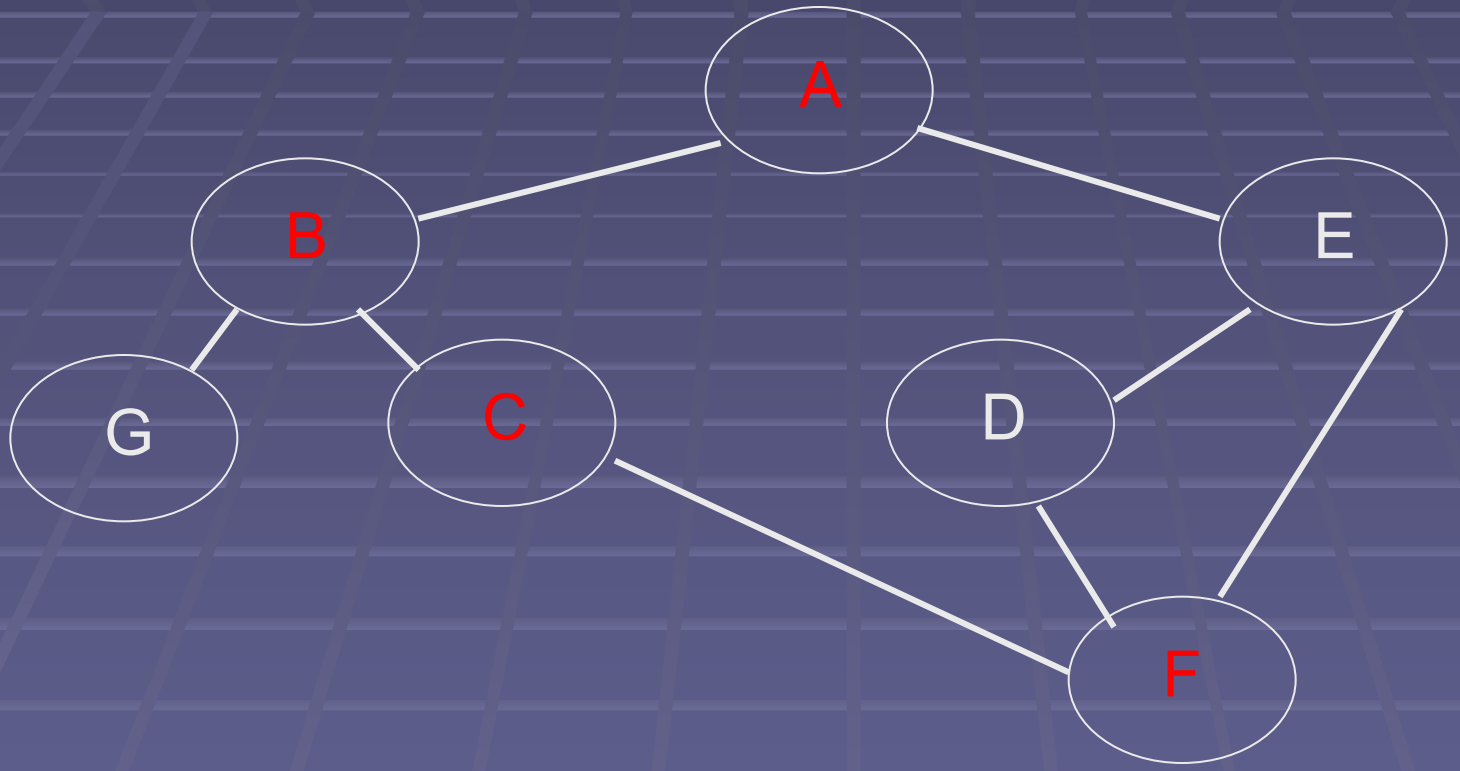
Expand A's adjacent vertices. Pick one (B). Mark it and re-visit.

Current: C



Now expand B, and visit its neighbor, C.

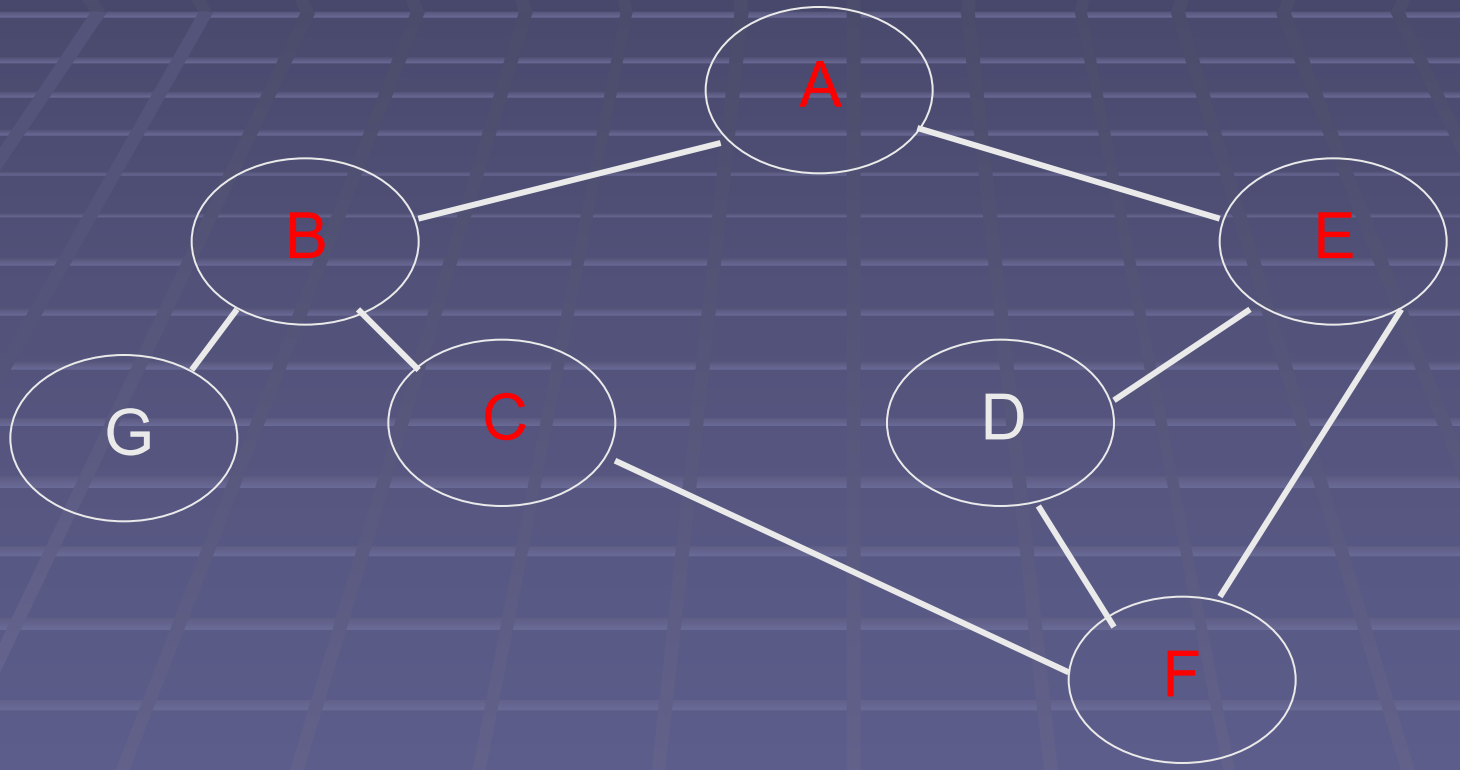
Current: F



Visit F.

Pick one of its neighbors, E.

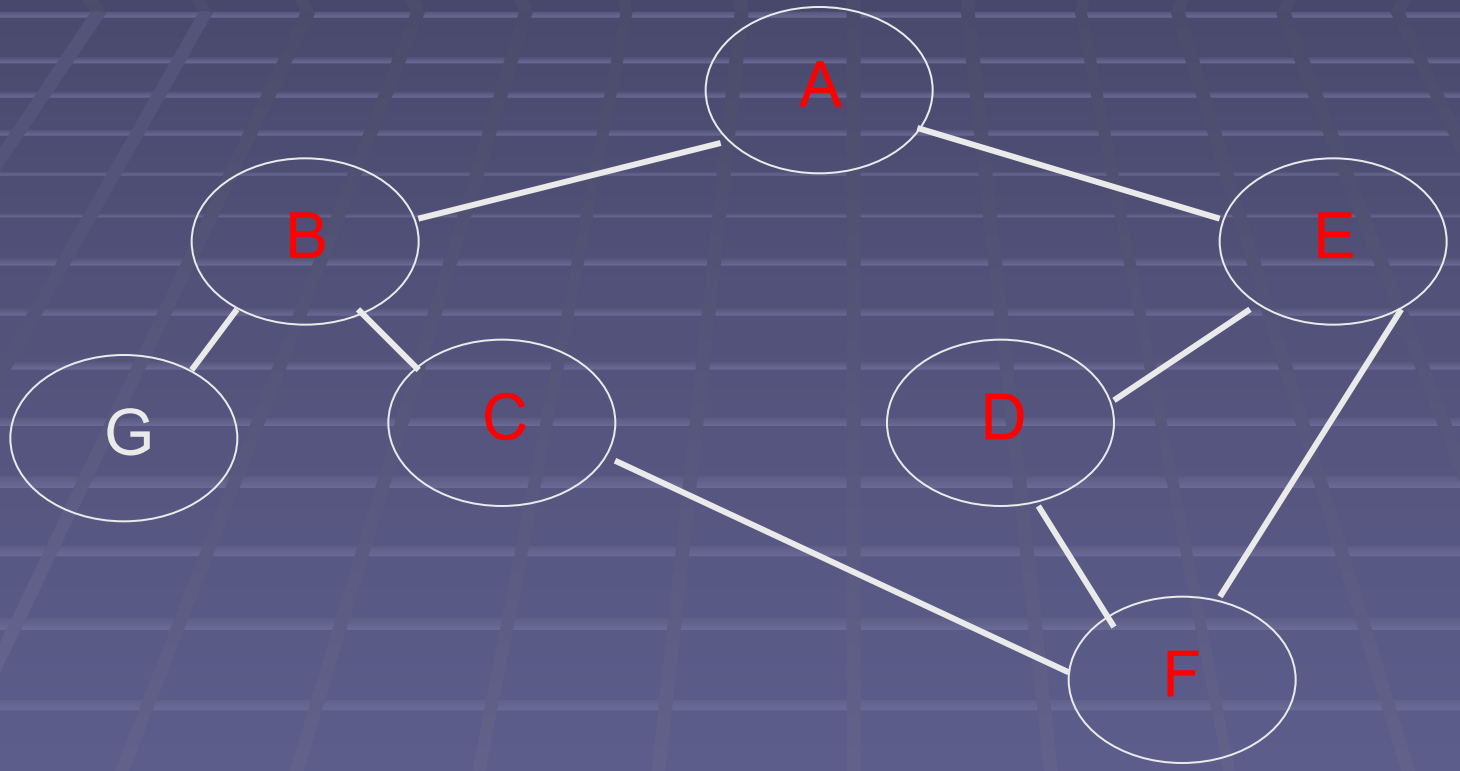
Current: E



E’s adjacent vertices are A, D and F.

A and F are marked, so pick D.

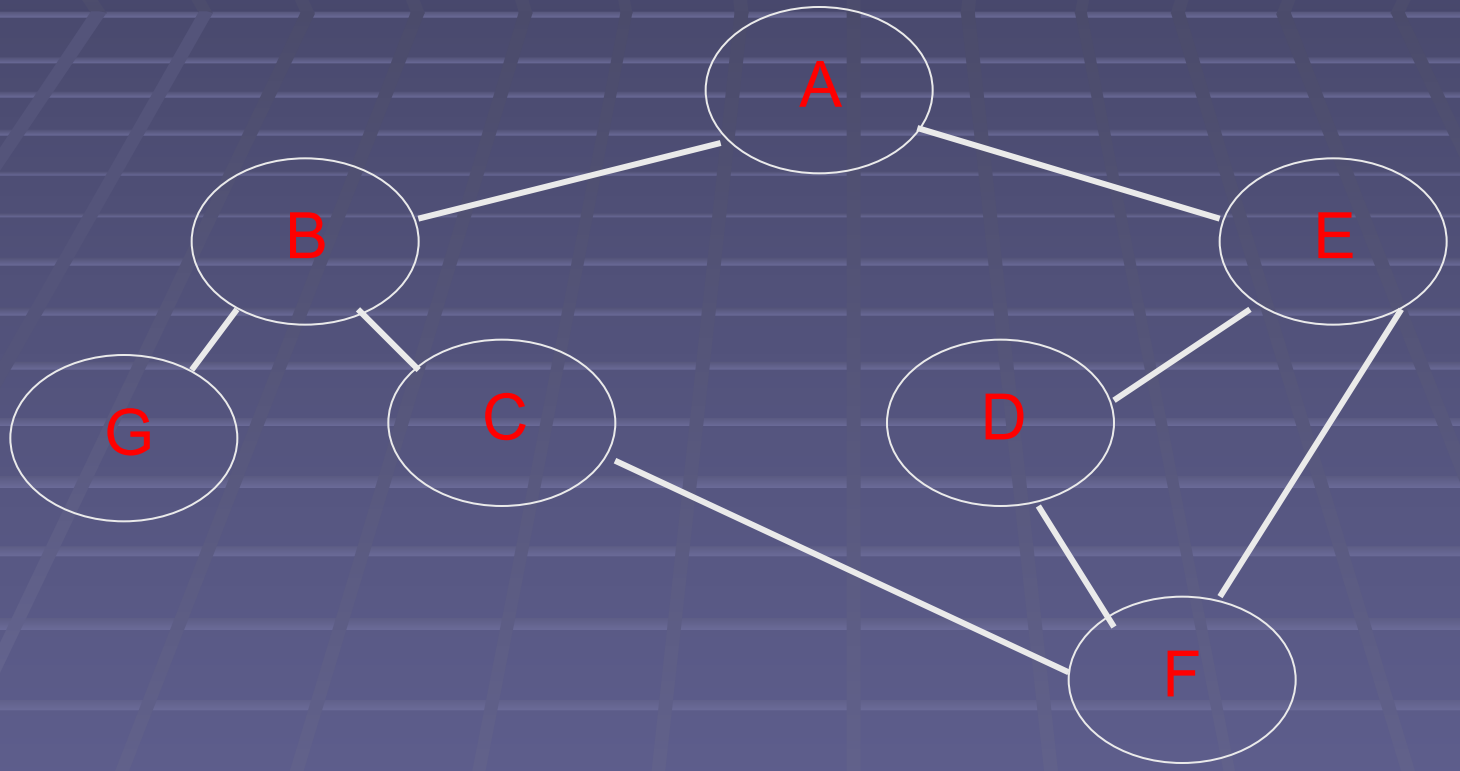
Current: D



Visit D. No new vertices available. Backtrack to

E. Backtrack to F. Backtrack to C. Backtrack

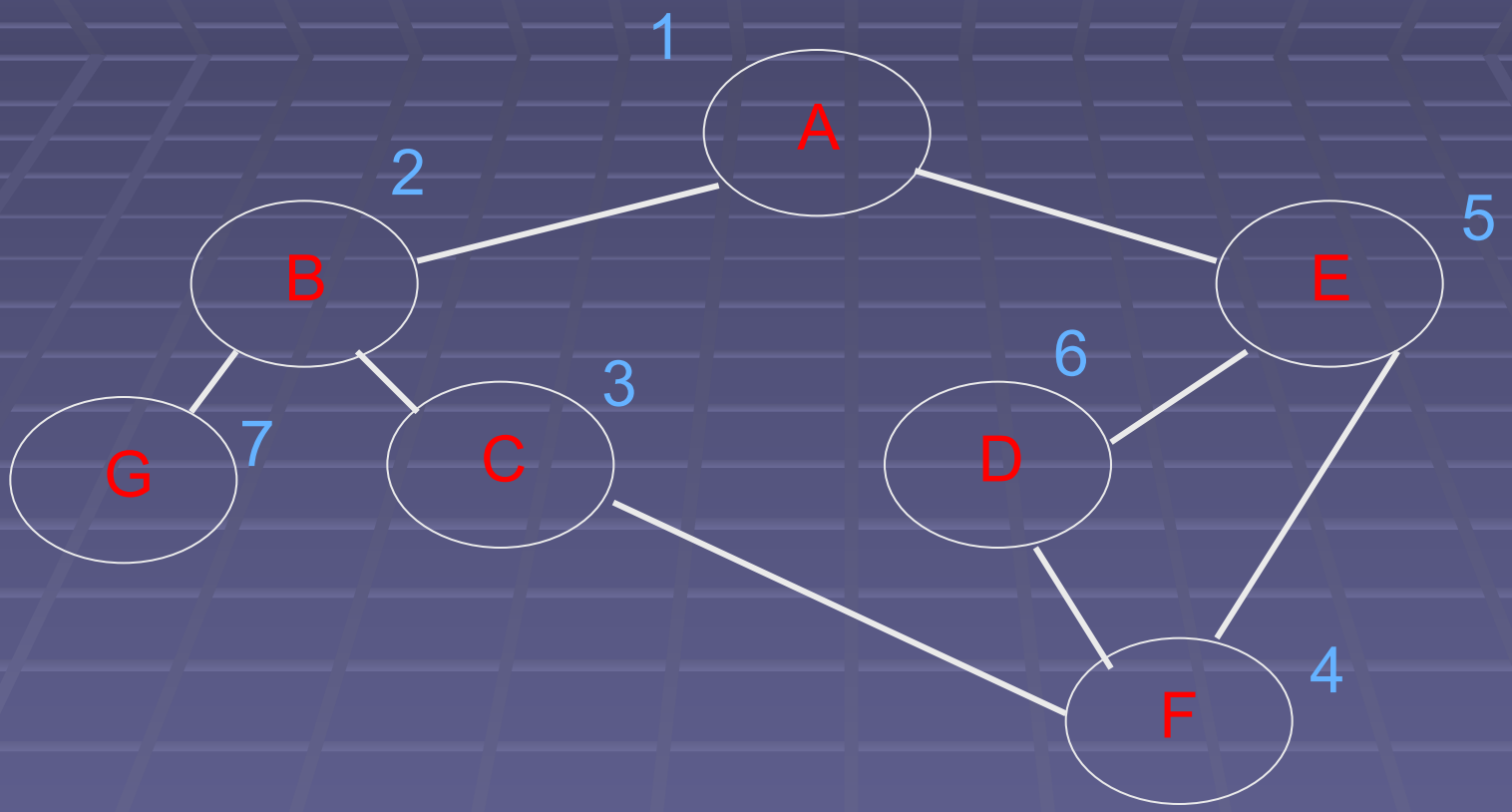
Current: G



Visit G. No new vertices from here. Backtrack to

B. Backtrack to A. E already marked so no

Current:



Done. We have explored the graph in order:

A B C F E D G